Solution: Is Graph Bipartite?

Problem Statement

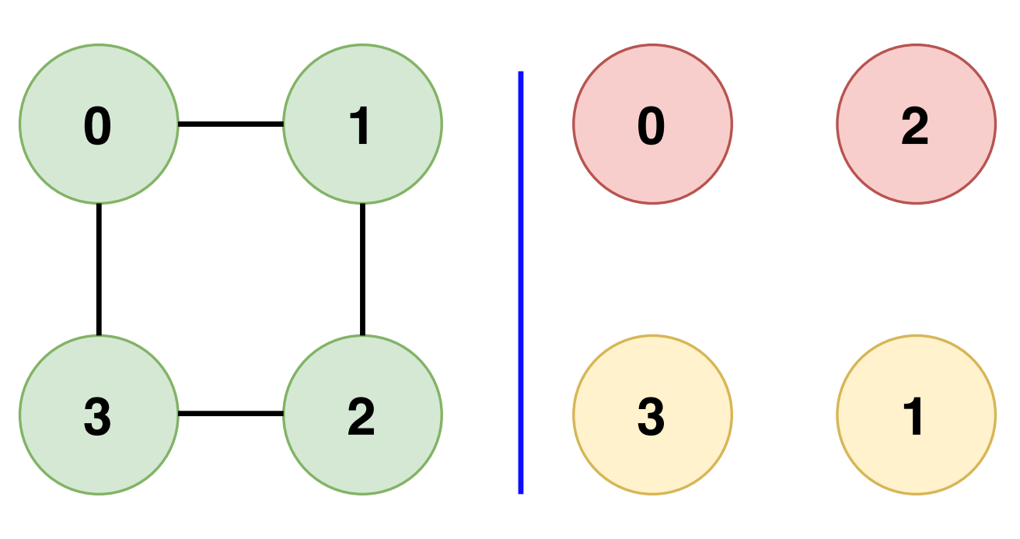
Given a 2D array `graph[][]`, representing the undirected graph, where `graph[u]` is an array of nodes that are connected with node `u`.

Determine whether a given undirected graph is a bipartite graph.

The graph is a bipartite graph, if we can split the set of nodes into two distinct subsets such that no two nodes within the same subset are adjacent (i.e., no edge exists between any two nodes within a single subset).

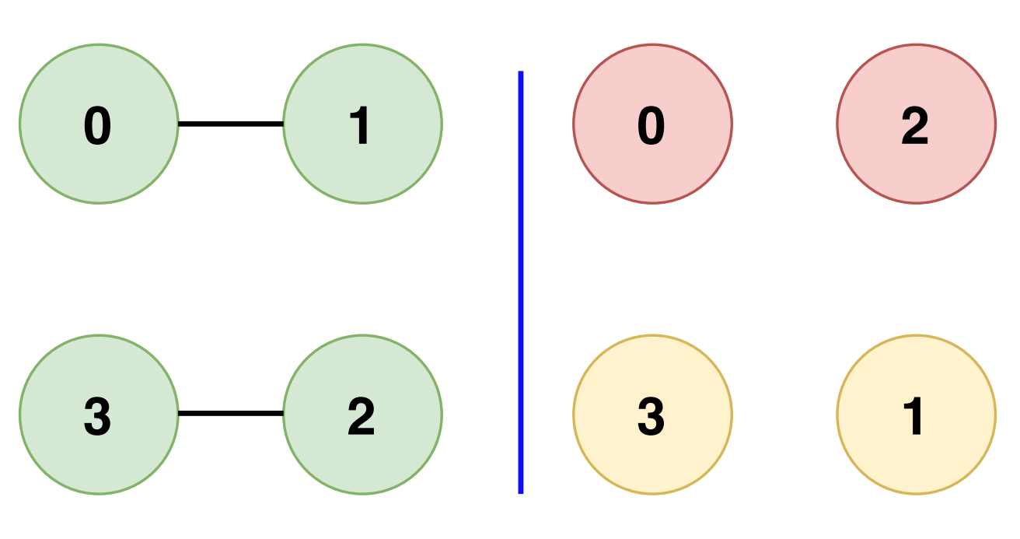
Examples

1. **Example 1**:
   - **Input**: `[[1,3], [0,2], [1,3], [0,2]]`



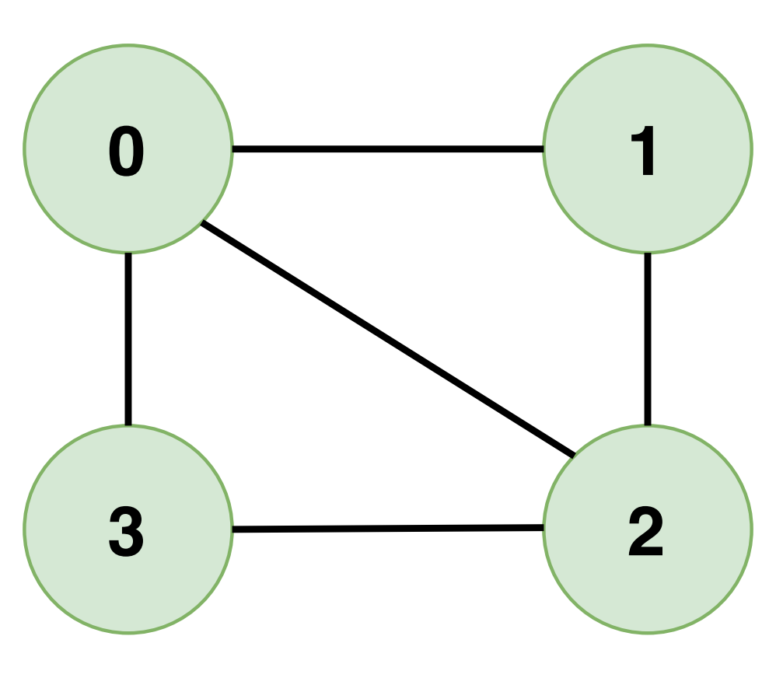
- **Expected Output**: `true`
- **Justification**: The nodes can be divided into two groups: {0, 2} and {1, 3}. No two nodes within each group are adjacent, thus the graph is bipartite.

2. **Example 2**:
   - **Input**: `[[1], [0], [3], [2]]`



- **Expected Output**: `true`
- **Justification**: The graph is a simple chain of 4 nodes. It's clearly bipartite as we can have {0, 2} and {1, 3} as the two subsets.

3. **Example 3**:
   - **Input**: `[[1,2,3],[0,2],[0,1,3],[0,2]]`



- **Expected Output**: `false`
- **Justification**: We found that edges (1, 2), (1, 3), and (2, 3) connect nodes within the same set, which violates the condition for a bipartite graph where each edge must connect nodes in different subsets. Thus, there's no way to divide this graph into two subsets that satisfy the bipartite condition.

Solution

To check if a graph can be bipartitioned, we employ the Union Find strategy. Initially, each node is treated as a distinct set, signifying that every node is in its own group. As we progress, each edge in the graph is examined. The pivotal aspect here is to determine whether the nodes connected by an edge are in different sets. If they are in the same set, it indicates a violation of the bipartition condition. For edges connecting nodes in separate sets, we perform a union operation, merging these sets. This step is crucial as it helps in grouping the nodes in a manner that allows for a potential bipartition.

Throughout this process, path compression flattens the structure of the sets, speeding up subsequent operations. The graph can be considered bipartite if, after evaluating all edges, no adjacent nodes are found in the same set. However, if any adjacent nodes are in the same set, the graph fails the bipartition test. This method effectively uses the concept of disjoint sets to determine if a given graph can be split into two groups without any overlapping edges.

Step-by-step algorithm

1. **Initialization**: We initialize the Union-Find structure to keep track of the connected components.
2. **Graph Processing**: We iterate through each edge in the graph. For each edge, we find the roots of the nodes it connects. If the roots are the same, it means there's a loop and the graph cannot be bipartite.
3. **Union Operations**: When we find that the roots are different, we perform a union operation to connect these nodes in our Union-Find structure. Additionally, we check that if the end vertices of an edge are in the same set, it's not bipartite.
4. **Checking Bipartiteness**: After processing all edges, if we don’t find any edge that connects nodes of the same set, the graph is bipartite.

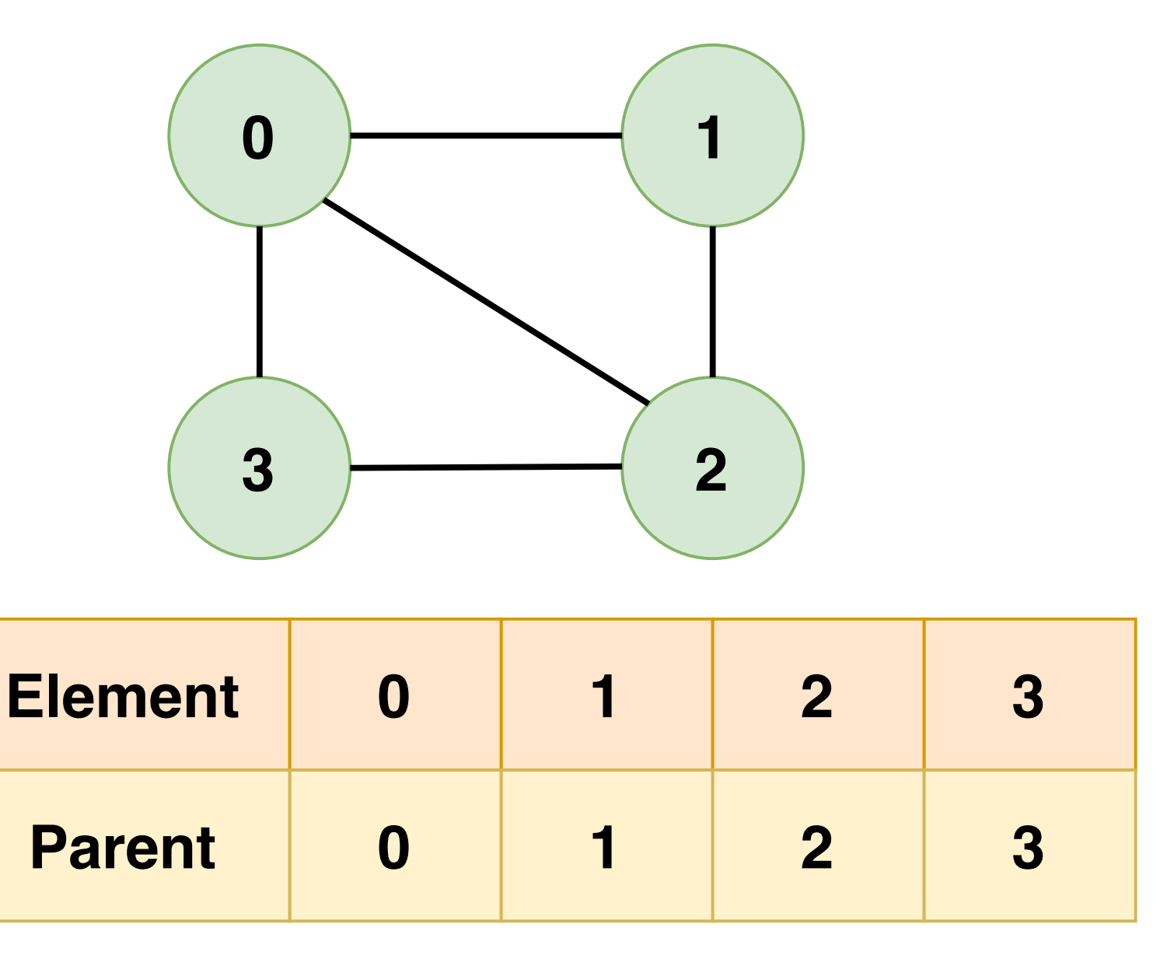
This algorithm works because a bipartite graph will never contain an odd-length cycle, and by checking that there is no edge connecting nodes of the same set, we are effectively checking for cycles that would violate the bipartite condition.

Algorithm Walkthrough

Let's consider the input [[1,2,3],[0,2],[0,1,3],[0,2]].

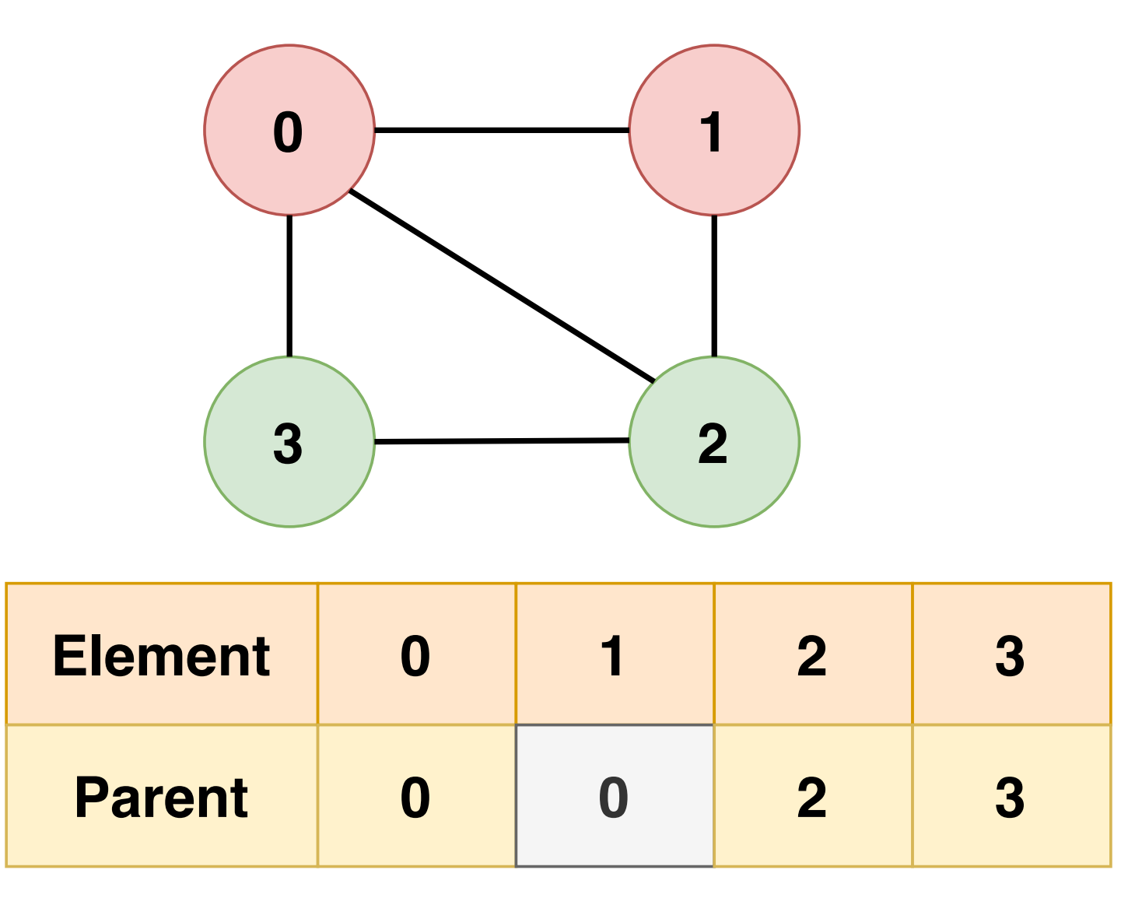
1. **Initialize Union-Find for 4 nodes.**
   - We start with 4 nodes (0, 1, 2, 3), each in its own set.

In the start



Element	0	1	2	3
Parent	0	1	2	3

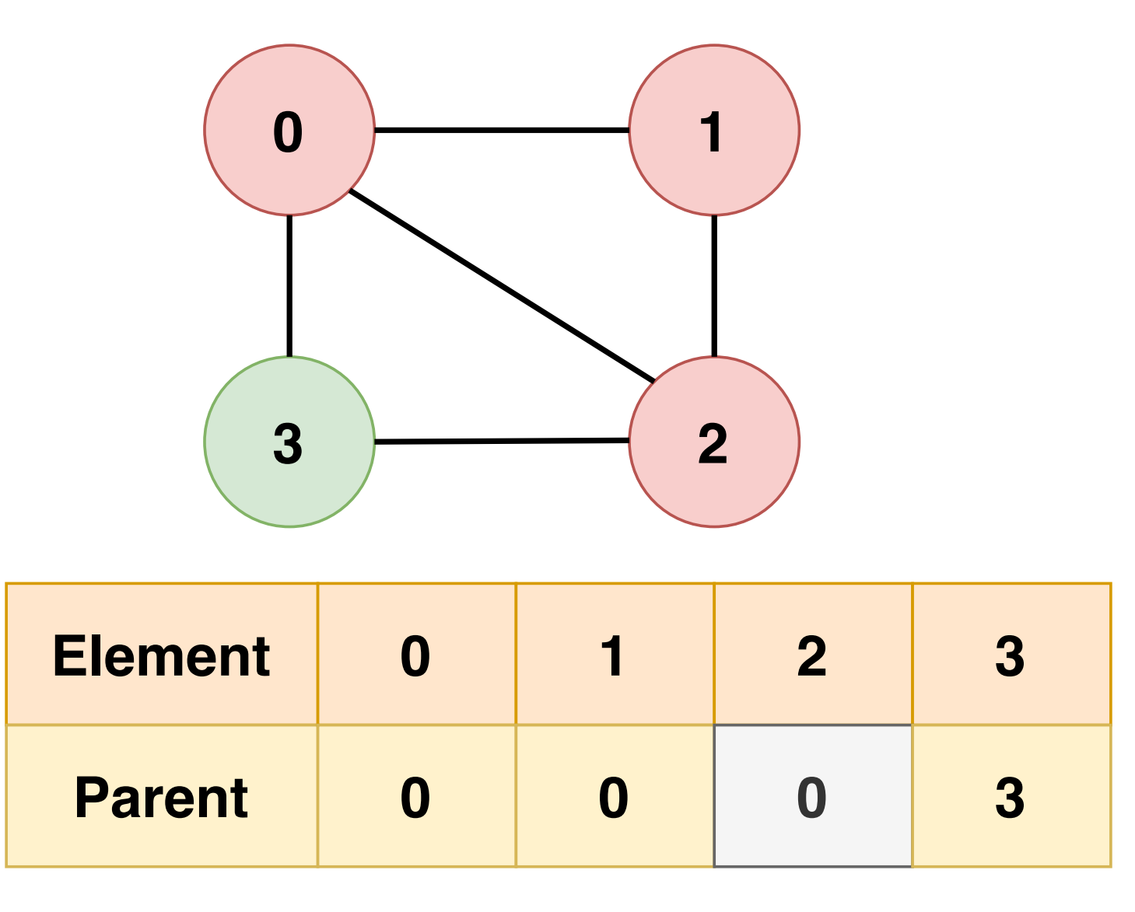
2. **Process edge (0, 1)**:
   - Roots are different (0 and 1), perform union.
   - Union set now has {0, 1}.



Element	0	1	2	3
Parent	0	0	2	3

Parent of 0 is 0 itself.
So, parent of 1 will be 0 after taking union.

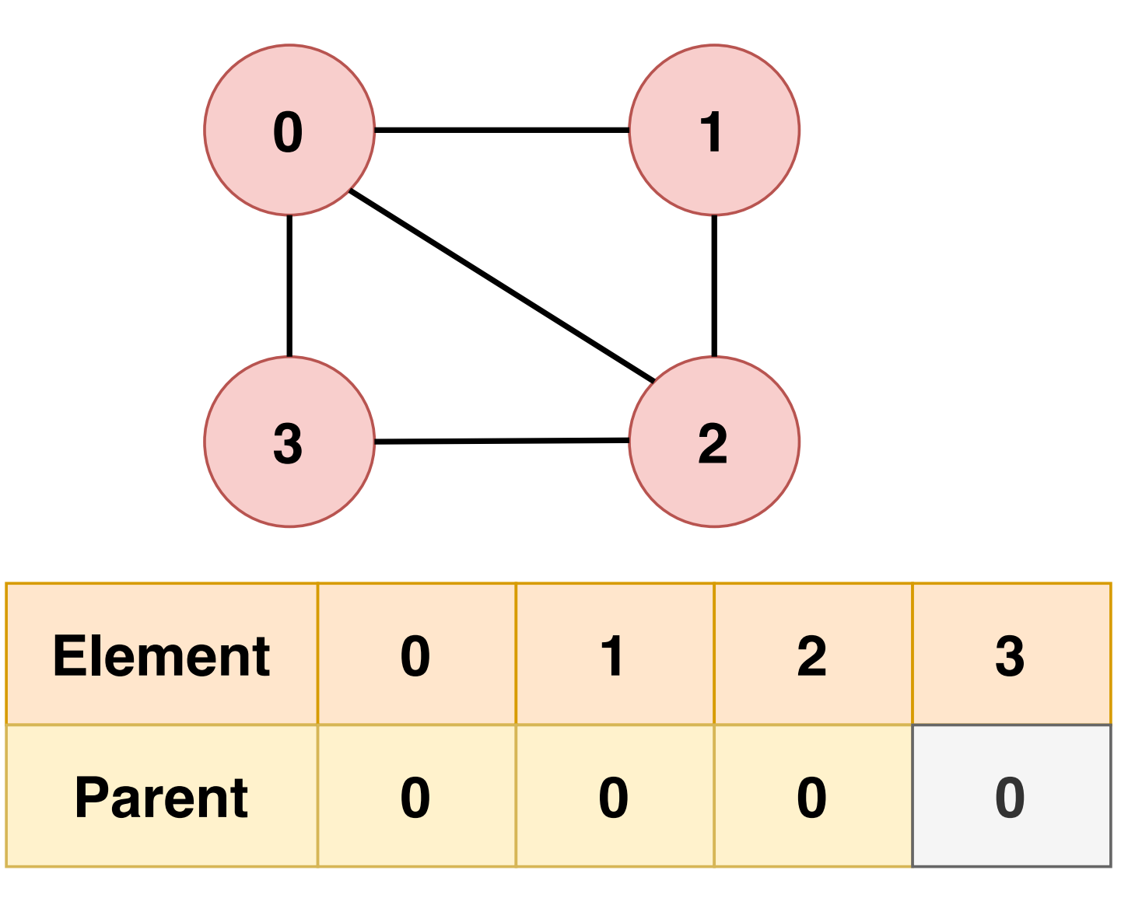
3. **Process edge (0, 2)**:
   - Roots are different (0 and 2), perform union.
   - Union set now has {0, 1, 2}.



Element	0	1	2	3
Parent	0	0	0	3

Parent of 0 is 0 itself.
So, parent of 2 will be 0 after taking union.

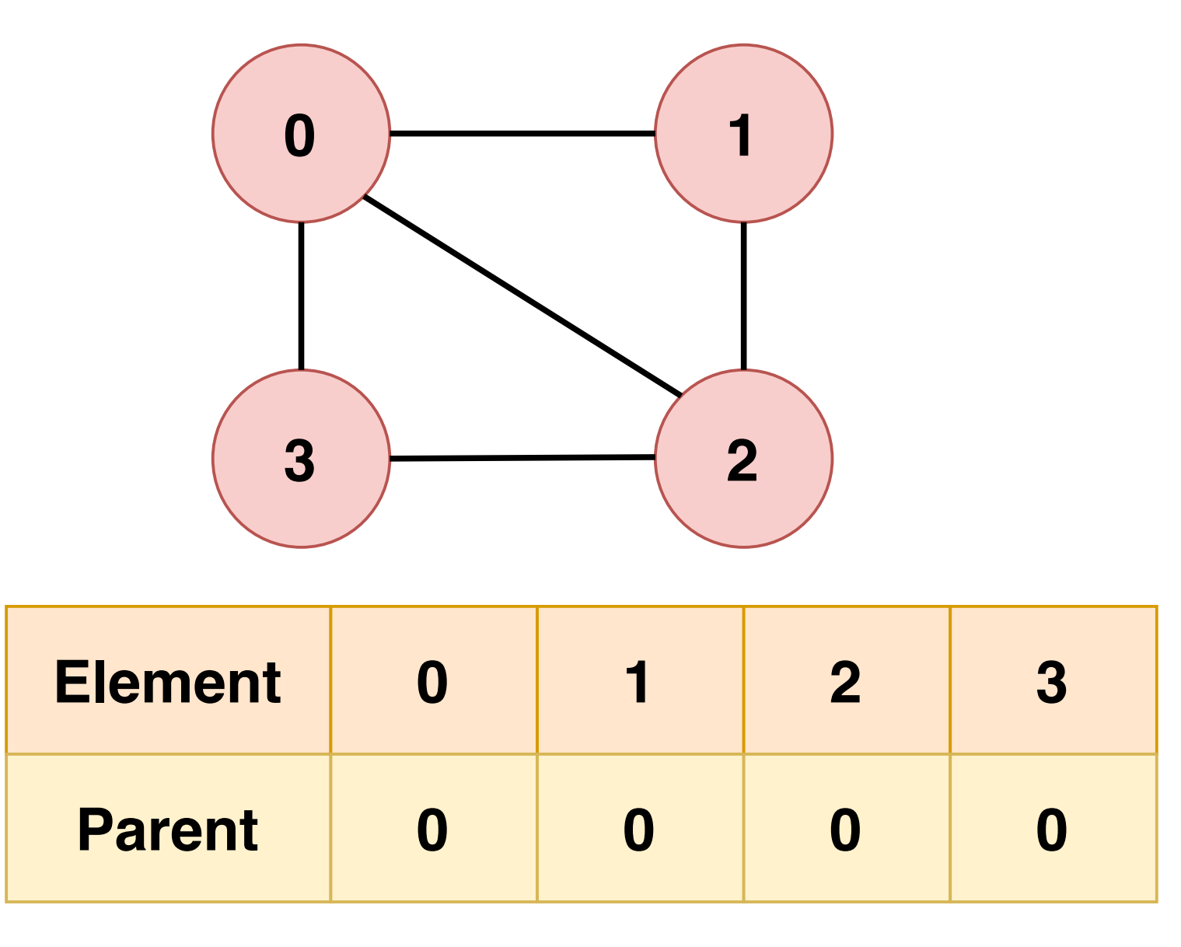
4. **Process edge (0, 3)**:
   - Roots are different (0 and 3), perform union.
   - Union set now has {0, 1, 2, 3}.



Element	0	1	2	3
Parent	0	0	0	0

Parent of 0 is 0 itself.
So, parent of 3 will be 0 after taking union.

5. **Process edge (1, 2)**:
   - Now roots are the same (both are in the set {0, 1, 2, 3}), which means there's an edge connecting the same set; hence, we detect a cycle.
   - At this point, we can already conclude that the graph is not bipartite, but let's continue for the sake of the walkthrough.

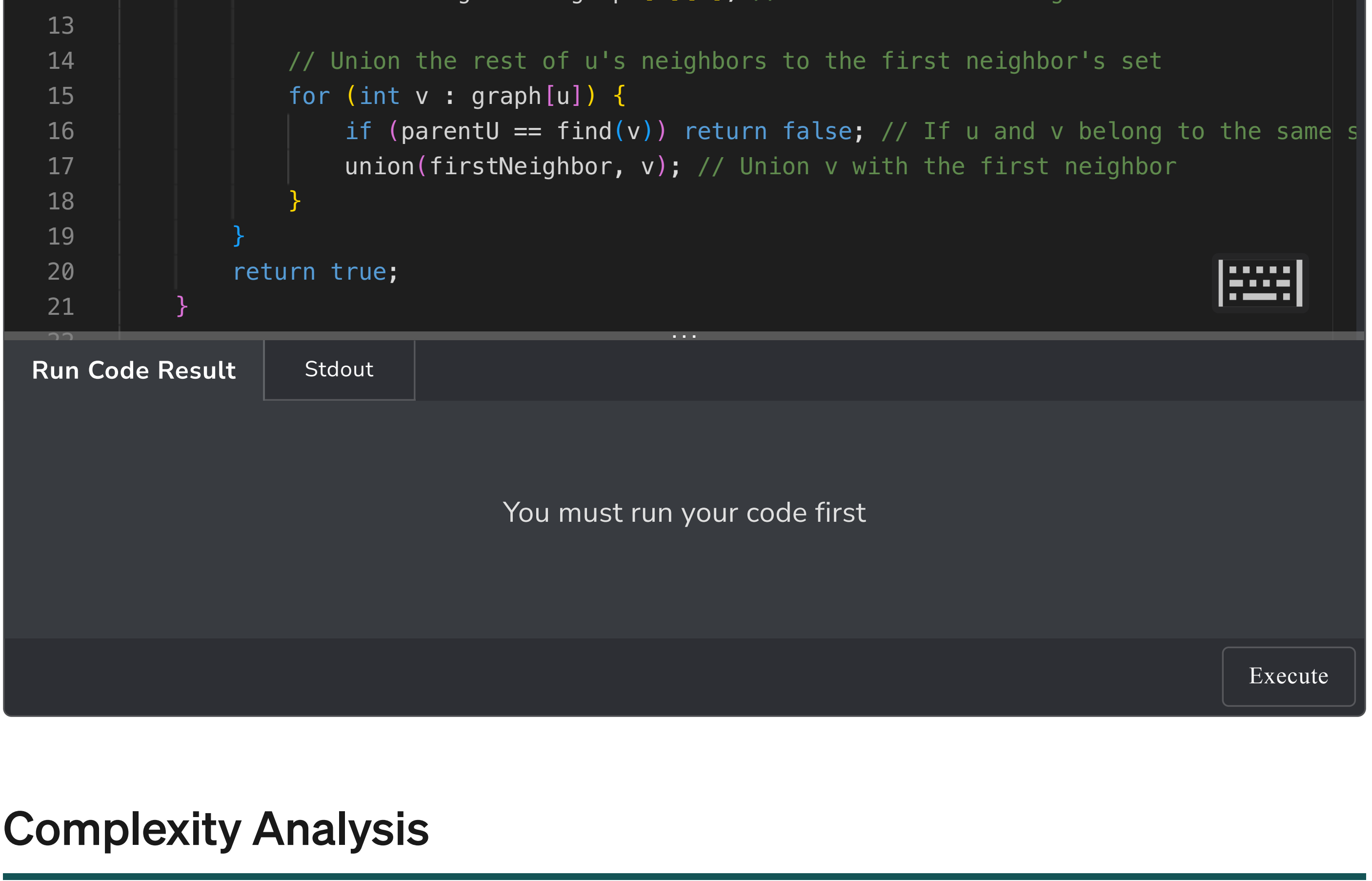


Element	0	1	2	3
Parent	0	0	0	0

Parent of 1 and 2 are same.
It means cycle is detected In the graph.
So, return false.

6. **Process edge (1, 3)**:
   - Roots are the same (both are in the set {0, 1, 2, 3}), further confirming that the graph is not bipartite.
7. **Process edge (2, 3)**:
   - Roots are the same (both are in the set {0, 1, 2, 3}), which again indicates an edge within the same set.

Code



```java
public class Solution {
    private int[] parent;

    public boolean isBipartite(int[][] graph) {
        parent = new int[graph.length];
        for (int i = 0; i < graph.length; i++) {
            parent[i] = i;
        }
        for (int u = 0; u < graph.length; u++) {
            if (graph[u].length == 0) continue; // No edges for this node
            int parentU = find(u); // Find the parent set for u
            int firstNeighbor = graph[u][0]; // Take the first neighbor

            // Union the rest of u's neighbors to the first neighbor's set
            for (int v : graph[u]) {
                if (parentU == find(v)) return false; // If u and v belong to the same s
                union(firstNeighbor, v); // Union v with the first neighbor
            }
        }
        return true;
    }
```

You must run your code first

Complexity Analysis

Time complexity:

- Each `union` and `find` operation can be considered almost O(1) due to path compression.
- The main loop processes each edge once, and there are E edges in the graph.
- Hence, the time complexity is O(E), where E is the number of edges in the graph.

Space complexity:

- We use an array to represent the union-find structure. This array has a size equal to the number of vertices V.
- Hence, the space complexity is O(V), where V is the number of vertices in the graph.

← Previous
Is Graph Bipartite? (Medium)

Next →
Path With Minimum Effort (Medium)

☐ Mark as Completed